



Universidad de Jaén

Bloque 4

Diseño de Software



EPS
Escuela Politécnica
Superior de Jaén

Tema 11. Diseño Detallado

L. Alfonso Ureña López Eugenio Martínez Cámara

Departamento de Informática
Universidad de Jaén

Fundamentos de Ingeniería del Software
Grado en Ingeniería Informática
2º Curso

Contenidos

- 1 Objetivos
- 2 Introducción
- 3 Especificación de atributos y operaciones
 - Tareas a realizar
 - Tipos de datos de los atributos
 - Especificación de clases: atributos
 - Especificación de clases: operaciones
- 4 Agrupando operaciones y atributos en clases
 - Principios generales
 - Interfaces
 - Acoplamiento y cohesión
 - Principio de Sustitución de Liskov (LSP)
- 5 Diseño de asociaciones
 - Tipos de asociaciones
 - Clases colección
- 6 Restricciones de integridad
- 7 Diseño de operaciones

Objetivos

- Comprender cómo diseñar atributos
- Comprender cómo diseñar operaciones
- Comprender cómo diseñar clases
- Comprender cómo diseñar asociaciones
- Conocer el impacto de las restricciones de integridad en el diseño

Introducción

- El **diseño detallado** o **diseño de objetos** guarda relación con el **diseño de clases, objetos y sus interacciones**
- Se ocupa de:
 - La especificación de los tipos de atributos
 - Cómo funcionan las operaciones
 - Cómo se vinculan entre sí los distintos objetos

Tareas a realizar

- Decidir el **tipo de datos** de **cada atributo**
- Decidir cómo manejar **atributos derivados**
- Adición de **operaciones primarias**
- Definir la **firma de operaciones**, incluyendo los tipos de parámetros
- Decidir la **visibilidad** de los atributos y operaciones

Tipos de datos de los atributos

Un tipo de dato de atributo se declara en un diagrama de clases UML con la sintaxis:

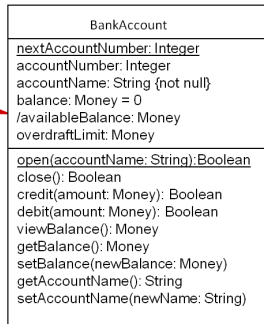
```
<property> ::= [<visibility>] ['/' ] <name> [ ':' <prop-type> ]  
['[' <multiplicidad> ']' ] ['=' <default> ]  
['{' <property-string> [',' <property-string>]* '}' ]
```

donde

- **name** es el nombre del atributo
- **prop-type** es el tipo de dato
- **default** es el valor que se le asigna al atributo cuando el objeto se crea por primera vez
- **property-string** describe una propiedad del atributo, tal como constante o fijo

Especificación de clases: atributos

Muestra un
atributo
derivado



**Clase BankAccount
con los tipos de datos de
los atributos incluidos**

Especificación de clases: atributos

- El atributo balance de la clase BankAccount se puede declarar con un valor inicial de cero utilizando la sintaxis
`balance:Money = 0.00`
- Los atributos se pueden restringir para que no puedan ser nulos

`accountName:String {not null}`

- Se puede indicar la posibilidad de ser nulo mediante la multiplicidad

`qualification:String[0..10]`

Operaciones Primarias



Constructor

Operación para crear una instancia de una clase. Por lo general, tiene el mismo nombre que la clase



Destructor

Operación para eliminar una instancia de una clase de memoria. C# y C++ tienen destructores explícitos con el mismo nombre que la clase con una virgulilla al principio, por ejemplo ~BankAccount

Operaciones Primarias



Get Operation

Operación para obtener el valor de un atributo, también conocido como un Accessor



Set Operation

Operación para establecer el valor de un atributo, también conocido como Mutator

Firma de las operaciones

- La **firma** de una operación viene determinada por el **nombre** de la operación, el **número y tipo** de sus **parámetros** y el **tipo** de valor **devuelto**, si es que hay alguno. La sintaxis para una operación:

```
[<visibility>] <name> ' (' [<parameter-list>] ') '  
[ [' : ' <return-type> ] [ ' [ ' <multiplicity> ' ] ' ]  
[ ' { ' <property-string> [ ' , ' <property-string> ] * ' } ' ] ]
```

- La **parte obligatoria** es el **nombre** de la operación seguida por un par de paréntesis
- La **lista de parámetros** es **opcional**. Si existe, se trata de una lista de nombres de parámetros y tipos separados por comas:

```
<parameter-list> ::= <parameter> [ ' , ' <parameter> ] *  
<parameter> ::= [ <direction> ] <parameter-name> ' : ' <type-expression>  
[ ' [ ' <multiplicity> ' ] ' ] [ ' = ' <default> ]  
[ ' { ' <property-string> [ ' , ' <property-string> ] * ' } ' ]
```

Ejemplo de uso

- BankAccount puede tener asociada una **operación credit** que se muestra en el diagrama:

```
credit (amount: Money) : Boolean
```

- El mensaje **credit** podría ser enviado por una instancia de BankAccount mediante el siguiente **formato**:

```
creditOK = accObject.credit (500.00)
```

- Se utiliza la **sintaxis de Java** para manejar **excepciones**, como se muestra en el siguiente fragmento de código:

```
try{  
    accObject.credit (500.00);  
} catch (UpdateException){//some error handling;}
```

Ejemplo de uso

BankAccount
<u>nextAccountNumber: Integer</u> accountNumber: Integer accountName: String {not null} balance: Money = 0 /availableBalance: Money overdraftLimit: Money
<u>open(accountName: String): Boolean</u> close(): Boolean credit(amount: Money): Boolean debit(amount: Money): Boolean viewBalance(): Money getBalance(): Money setBalance(newBalance: Money) getAccountName(): String setAccountName(newName: String)

Clase BankAccount con
firma de operaciones
incluidas

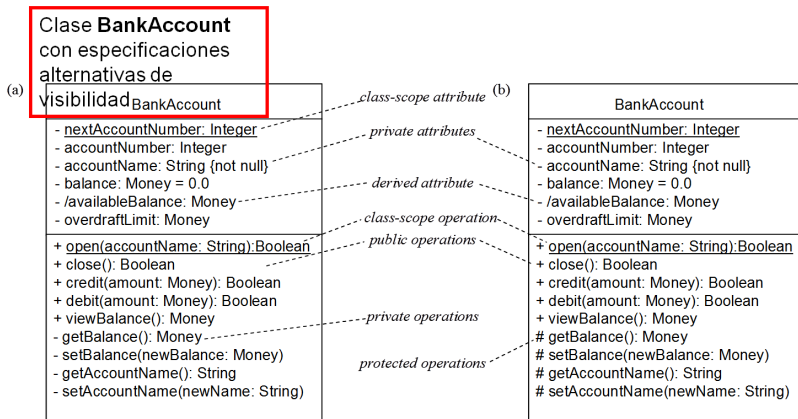
¿Qué operaciones mostramos?

- Mostrar las **operaciones primarias** donde sea necesario
- Mostrar sólo los **constructores** donde tengan un significado especial
- **Varios niveles de detalle** en las **diferentes etapas** del ciclo de desarrollo

Visibilidad

Símbolo de visibilidad	Visibilidad	Significado
+	Public	Cualquier instancia de cualquier clase podrá acceder a la característica (una operación o atributo)
-	Private	La característica sólo podrá ser utilizada por una instancia de la clase que la incluye
#	Protected	La característica podrá ser usada por instancias de la clase que la incluye o por una subclase o descendiente de dicha clase
~	Package	La característica es directamente accesible sólo por las instancias de una clase perteneciente al mismo paquete.

Visibilidad: ejemplo



Principios generales

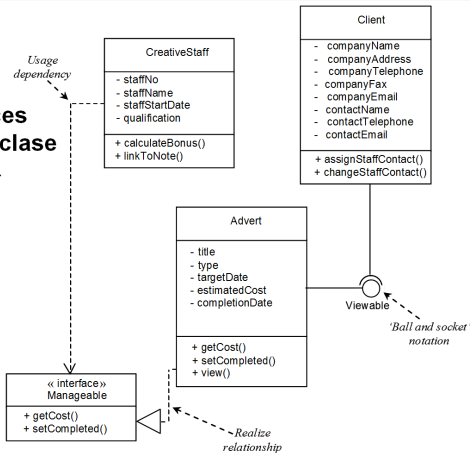
- **Comprobar** que las **responsabilidades** se han asignado a la **clase adecuada**
- Definir o usar **interfaces** para agrupar juntos los comportamientos estándar bien definidos
- Aplicar los conceptos de **acoplamiento y cohesión**
- Aplicar el **Principio de Sustitución de Liskov**

Interfaces

- UML utiliza **dos notaciones** para mostrar las **interfaces**:
 - El **icono de círculo pequeño** no muestra los detalles
 - El **icono de la clase estereotipada** con una lista de las operaciones soportadas
 - Normalmente, **sólo una** de ellas se utiliza en un diagrama
- La **relación *realize*** representada por la línea discontinua con una punta de flecha triangular, indica que la **clase *CreativeStaff*** (respecto a *Advert*) **soporta** al menos las **operaciones** listadas en la **interfaz**

Interfaces

Interfaces para la clase Advert



Definiciones



Acoplamiento

- Describe el **grado de interconexión** entre los **componentes** diseñados
- Queda reflejado por el **número de enlaces** que presenta un **objeto** y por el grado de **interacción** del objeto **con otros objetos**



Cohesión

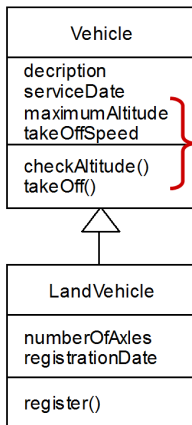
- Es una **medida del grado** en que un elemento contribuye a un **objetivo concreto**
- Los conceptos de acoplamiento y cohesión **se complementan mutuamente**
- **Coad y Yourdon** (1991) sugieren varias maneras en que el **acoplamiento** y la **cohesión** pueden aplicarse dentro de un **enfoque orientado a objetos**

Acoplamiento de interacción

- Es una **medida** del **número de tipos de mensajes** que un **objeto envía** a otros objetos y el **número de parámetros que se pasan** con este tipo de mensajes
- **Deben mantenerse al mínimo** para reducir la posibilidad de cambios a través de las interfaces y hacer más fácil la reutilización

Acoplamiento de herencia

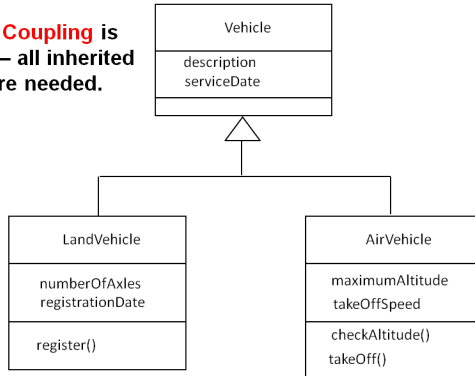
Acoplamiento de herencia
describe el grado con el que
una subclase necesita
realmente las funciones que
hereda de su clase base



Poor inheritance
coupling as unwanted
attributes and
operations are
inherited

Acoplamiento de herencia

Inheritance Coupling is better here – all inherited attributes are needed.



Tipos de cohesión



Cohesión de operación

- Mide el grado con el que se centra una operación sobre un único requisito funcional



Cohesión de clase

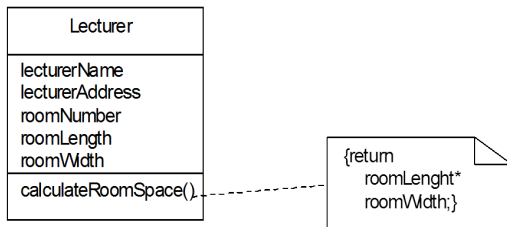
- Refleja el grado con el que una clase se centra en un único requisito



Cohesión de especialización

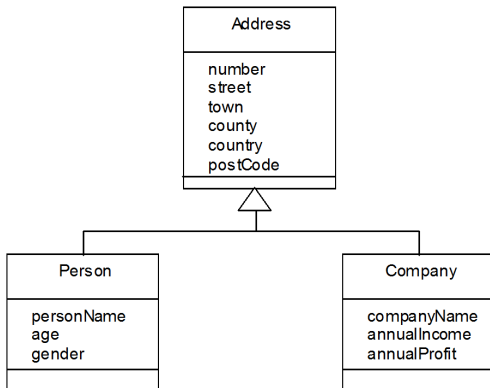
- Aborda la cohesión semántica de las jerarquías de herencia

Cohesión de operación

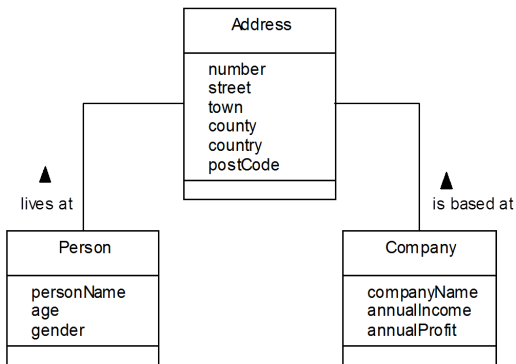


Buena cohesión de operación pero pobre cohesión de clase

Pobre cohesión de especialización



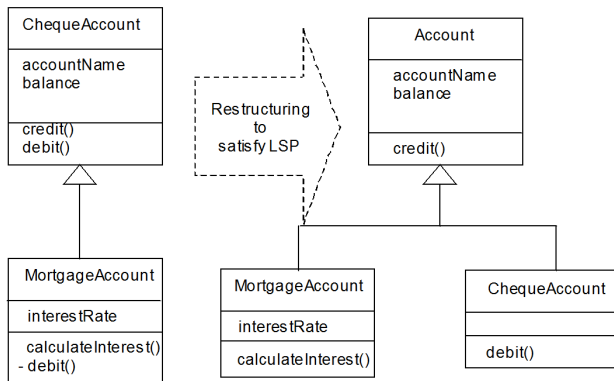
Estructura mejorada usando la clase Address



Principio de Sustitución de Liskov (LSP)

- Establece que, en la **interacción entre objetos**, debe ser posible **tratar** a un **objeto derivado como** si fuera un **objeto base** sin problemas de integridad
- Si el principio **no se aplica**, entonces puede **violarse la integridad del objeto derivado**

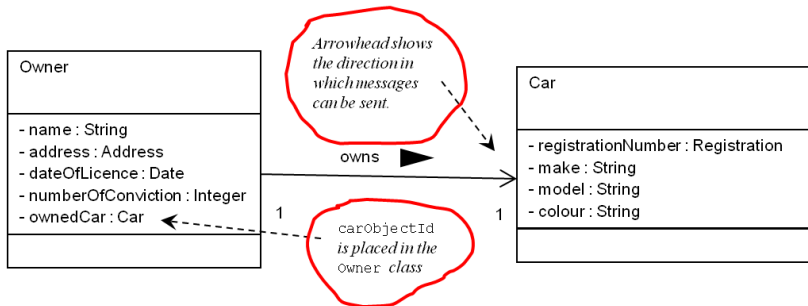
Aplicación del LSP



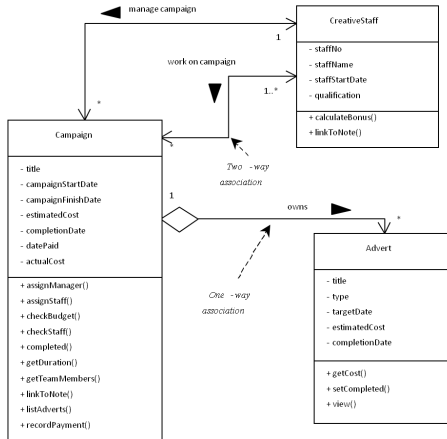
Asociación bidireccional

- Una **asociación** que tiene que soportar el **paso de mensajes en ambas direcciones** es una asociación bidireccional
- Una asociación bidireccional, se indica con **flechas en ambos extremos**
- **Minimizar** el número de **asociaciones bidireccionales** mantiene el **acoplamiento** entre objetos al **mínimo**

Asociación unidireccional una a una



Fragmento de un diagrama de clases



Clases colección

- Las **clases colección** se pueden diseñar para proporcionar un **apoyo adicional** para navegar por los objetos
- Las clases colección se utilizan para **mantener los identificadores de los objetos** cuando el paso de mensajes requiere de una **asociación de uno a muchos**
- Los **lenguajes orientados a objetos** soportan esta característica. Por ejemplo, en **Java** se puede aplicar un **iterador** a una instancia de la clase colección para permitir **iterar** por todos los elementos de la **colección**

Asociación uno a muchos con clase colección

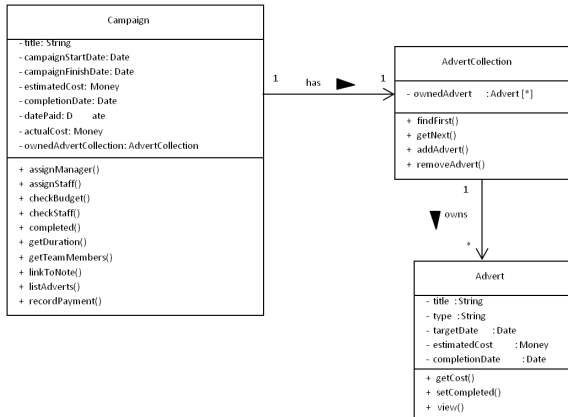
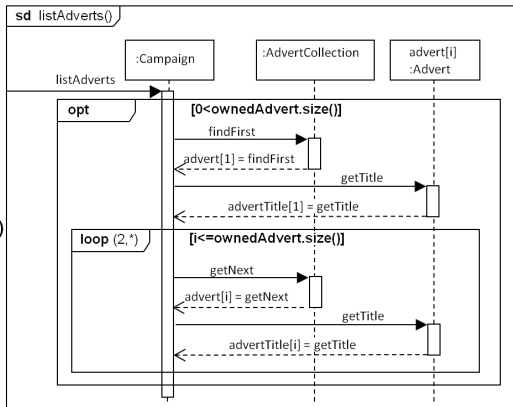
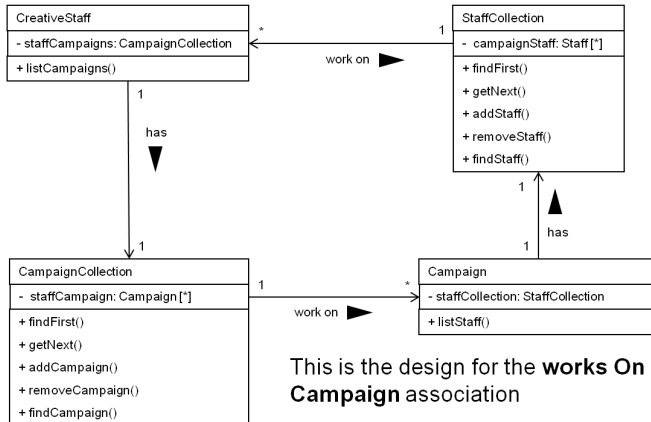


Diagrama de secuencia para *listAdverts()*

Este **diagrama de secuencia** muestra la **interacción** cuando se usa una **clase colección** (diapositiva anterior)



Asociación bidireccional muchos a muchos



Restricciones de integridad



Integridad referencial

- Asegura que un identificador de objeto referenciado dentro de otro objeto se refiere a un objeto que realmente existe



Restricciones de dependencia

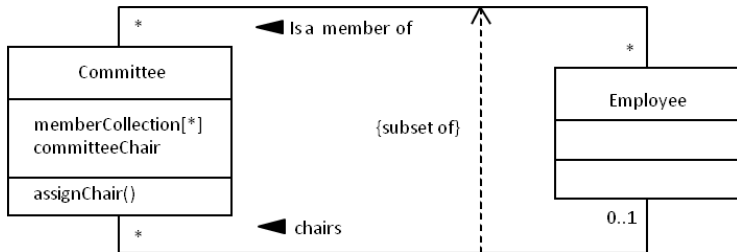
- Aseguran que las dependencias de atributos, donde un atributo puede ser calculado a partir de otros, son consistentes



Integridad de dominio

- Asegura que los atributos únicamente tomen valores adecuados al dominio en el que están definidos

Restricciones entre asociaciones



Diseño de operaciones

- Son varios los **factores** que limitan el **diseño de algoritmos**:
 - El coste de la implementación
 - Las restricciones de rendimiento
 - Los requisitos de precisión
 - Las capacidades de la plataforma de implementación

Diseño de operaciones

- *Rumbaugh et al. (1991)* sugieren tener en cuenta los siguientes **factores** para **elegir** entre **diseños alternativos** de **algoritmos**:
 - La **complejidad computacional**
 - **Facilidad de la implementación y de comprensión**: suele ser prudente sacrificar parte del rendimiento para simplificar la implementación
 - **Flexibilidad**: la mayoría de los sistemas software están sujetos a cambios, y deben diseñarse los algoritmos teniendo en cuenta este principio
 - **Puesta a punto del modelo de objetos**: la realización de ciertos ajustes en el modelo de objetos puede simplificar el algoritmo

Bibliografía



S. Bennet et al.

Análisis y Diseño Orientado a Objetos de Sistemas Usando UML. 3ª Ed.

McGraw-Hill, 2007